

Enterprise Network Defense & SOC Incident Response Lab

Blue Team / SOC Portfolio Case Study

Prepared by: Assem

Version 1.0 | September 2026

Project Focus

An enterprise-style security monitoring lab integrating segmented network design, custom network intrusion detection, packet analysis, SIEM investigation, Linux endpoint telemetry, and firewall-based incident response. The project demonstrates an end-to-end SOC workflow through controlled attack simulations, evidence correlation, containment, recovery, and detection tuning.

Core stack: pfSense | Snort 3 | Wireshark | Splunk Enterprise | Wazuh | Auditd | Docker | DVWA | Kali Linux | Ubuntu Server

Detection | Investigation | Correlation | Containment | Recovery | Tuning

Portfolio purpose: SOC L1 / Blue Team interviews, GitHub evidence, and CV support

Executive Summary

This project documents the design, operation, investigation, and response workflow of an enterprise-style security monitoring lab built for Blue Team and SOC training. The environment combined network segmentation, intentionally vulnerable services, network intrusion detection, packet analysis, SIEM investigation, endpoint telemetry, file integrity monitoring, and firewall-based incident response into one coherent workflow rather than a collection of isolated exercises.

The core architecture placed a Kali Linux attacker workstation on a LAN segment, an Ubuntu Server hosting Dockerized DVWA inside a DMZ, and a dedicated Wazuh security server on a separate SOC network. pfSense provided routing, stateful filtering, segmentation, and later incident containment. Snort generated network detections, Wireshark provided packet-level validation, Splunk centralized and visualized Snort telemetry, while Wazuh and Auditd provided endpoint evidence and file integrity monitoring.

Three controlled incident scenarios were investigated: DVWA command injection, SSH password guessing / brute-force activity, and a harmless cron-based persistence simulation. The command-injection case was validated through network-to-endpoint correlation: Snort detected the malicious whoami payload at 15:43:00.558, and Auditd recorded /usr/bin/dash execution approximately 0.104 seconds later, followed by ping and whoami under the www-data context. Wazuh converted the endpoint telemetry into rule 80792 alerts. This evidence supported a high-confidence conclusion that the application-layer attack resulted in actual command execution in the controlled target environment.

The project also included targeted containment through pfSense, recovery verification, and detection engineering. A generic repeated-request Snort rule was refined into a POST-specific /login.php detection and validated against both benign GET traffic and suspicious repeated POST traffic. The final result is a practical SOC portfolio case study demonstrating detection, validation, correlation, response, tuning, and documentation skills.

Key outcome

The strongest evidence in the project is not a single alert. It is the correlation of network telemetry, endpoint process execution, authentication logs, file integrity events, SIEM context, and firewall response into a defensible incident narrative.

Report Structure

The report follows one coherent sequence: architecture and segmentation -> attack simulation -> network and endpoint detection -> SIEM investigation -> correlation -> incident response -> detection tuning -> ATT&CK mapping and lessons learned.

1. Project Overview

The lab was built to simulate the workflow of a junior SOC analyst working across network, SIEM, and endpoint telemetry. Instead of treating each product as a standalone tool, the project connected the controls so that an alert observed in one layer could be validated against evidence from another.

All attack activity was performed inside the owner-controlled local VMware environment against intentionally vulnerable or deliberately configured lab services. No production systems, external targets, or unauthorized accounts were involved.

2. Project Objectives

- Design a segmented network with clear LAN, DMZ, and SOC security zones.
- Generate controlled attacker behavior and observe it across multiple telemetry layers.
- Write and validate custom Snort detections rather than relying only on default signatures.
- Use Wireshark to verify packet-level behavior behind IDS alerts.
- Ingest IDS telemetry into Splunk and perform basic SOC-style investigation and visualization.
- Deploy Wazuh and Auditd for endpoint process visibility, authentication monitoring, and file integrity monitoring.
- Correlate network alerts with endpoint execution evidence to distinguish attempts from confirmed execution.
- Perform targeted containment and recovery through pfSense.
- Tune detections against benign and suspicious traffic to reduce false positives.
- Document the project as a concise technical portfolio case study suitable for SOC L1 interviews.

3. Lab Architecture

The architecture followed a simple enterprise-style model with separate functional zones. Kali represented an untrusted workstation on the LAN, Ubuntu/DVWA represented an exposed application workload in the DMZ, and Wazuh occupied a dedicated SOC network. pfSense acted as the central routing and policy enforcement point.

Zone	System / Role	Addressing	Purpose
WAN	pfSense em0	192.168.221.132/24 (VMware NAT)	Internet/NAT uplink for the lab
LAN	pfSense em1	192.168.10.1/24	User / attacker segment
LAN	Kali Linux	192.168.10.10/24	Controlled attacker and analyst workstation
DMZ	pfSense em2	192.168.20.1/24	Gateway for the vulnerable server segment
DMZ	Ubuntu / Docker / DVWA	192.168.20.10/24	Target server and application workload
SOC	pfSense em3 / OPT2	192.168.30.254/24	Gateway for monitoring infrastructure

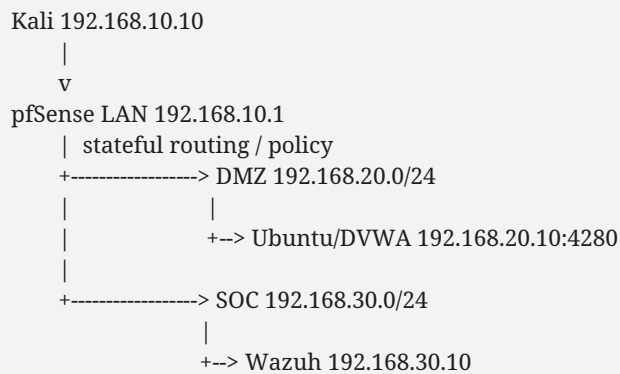
Zone	System / Role	Addressing	Purpose
SOC	Wazuh server	192.168.30.10/24	Manager, indexer, dashboard, endpoint security monitoring

Architecture principle

The project intentionally separated attacker, target, and monitoring roles. This made rule placement, traffic direction, and incident containment observable and explainable during investigation.

4. Network Topology and IP Addressing

Traffic from Kali to the Ubuntu/DVWA server entered pfSense through the LAN interface and was routed toward the DMZ. This interface direction later became important during incident response because the containment rule had to be placed on the ingress interface where the suspicious traffic entered the firewall.



5. Technologies and Security Controls

Technology	Role in the Lab	Security Value
pfSense	Routing, segmentation, firewalling, containment	Enforced zone boundaries and supported targeted response
Snort 3	Network IDS and custom detection rules	Detected scans, SQLi/command-injection indicators, repeated requests, DNS/ICMP activity
Wireshark	Packet analysis	Validated protocol behavior and malicious payloads at packet level
Splunk Enterprise	SIEM for Snort telemetry	Centralized logs, SPL searches, timelines, statistics, and dashboarding
Wazuh	Endpoint security monitoring / SIEM capabilities	Agent telemetry, authentication monitoring, FIM, alert correlation

Technology	Role in the Lab	Security Value
Auditd	Linux process auditing	Recorded execve events and execution context for endpoint validation
Docker + DVWA	Intentionally vulnerable application environment	Safe target for controlled web exploitation scenarios
Kali Linux	Controlled attacker workstation	Generated scans, HTTP attacks, and SSH password-guessing traffic
Ubuntu Server	Target host	Ran DVWA, Snort, Wazuh agent, and Auditd

6. Network Segmentation with pfSense

6.1 Segmentation Model

pfSense provided the central trust boundary between LAN, DMZ, WAN, and SOC networks. The DMZ was permitted outbound access while a higher-priority rule blocked DMZ-to-LAN traffic. Kali-to-DMZ traffic was initially permitted to support controlled testing. Rules were evaluated top-to-bottom, with the first matching rule determining the action.

This behavior was reinforced during troubleshooting when an OPT2 rule was accidentally configured as TCP-only. ICMP tests failed until the protocol was changed to Any, demonstrating the practical difference between transport protocols and ICMP as well as the importance of matching firewall rules to the intended validation traffic.

Rule placement lesson

For Kali -> Ubuntu traffic, the packets enter pfSense on LAN. Therefore the later containment rule was correctly placed on the LAN ruleset, above the broad LAN allow rule.

6.2 Stateful Filtering and Least Privilege

The firewall design relied on stateful inspection. Return traffic for permitted sessions was handled by state tracking, while new connections were evaluated against interface rules. The project used targeted rules rather than blocking the entire DMZ, which made the incident-response stage more representative of a controlled containment decision.

7. Target Environment

Ubuntu Server hosted the target application environment. DVWA was deployed with Docker Compose and exposed through TCP port 4280 on the Ubuntu host. MariaDB remained internal to the Compose environment. This provided a reproducible target while keeping the vulnerable application isolated from the public Internet.

The target host also ran Snort 3, the Wazuh agent, and Auditd. This combination made it possible to observe the same activity at multiple layers: network payloads through Snort, endpoint process execution through Auditd, and security alerts / FIM through Wazuh.

8. Network Detection with Snort

8.1 Detection Scope

Custom Snort rules were developed progressively as the lab expanded. The final ruleset covered representative traffic types rather than attempting to reproduce a full commercial signature feed. The emphasis was on understanding rule anatomy, validating detections, and later tuning noisy logic.

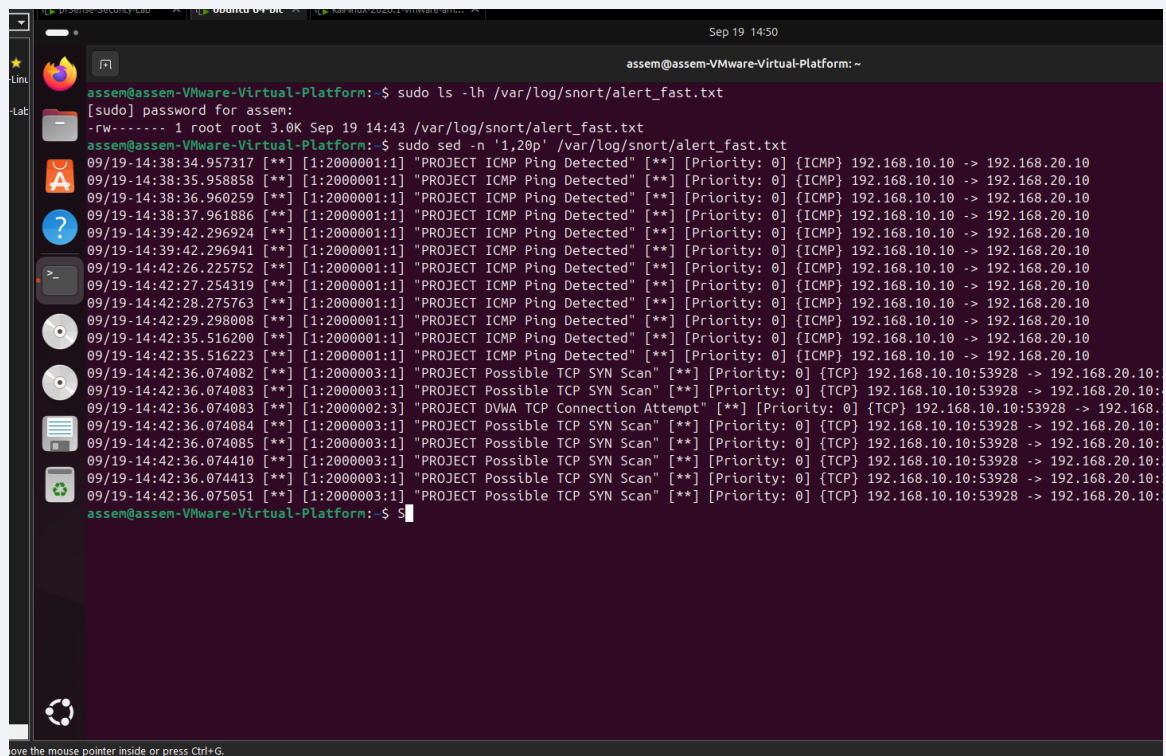
SID	Detection	Key Logic	Use in Project
2000001	ICMP ping	ICMP to HOME_NET	Basic reachability / IDS validation
2000002	DVWA TCP connection	SYN traffic to TCP/4280	Connection-attempt visibility
2000003	Possible SYN scan	SYN threshold by source	Scan behavior
2000004	Possible SQL injection	HTTP URI content: UNION + SELECT	Web exploitation indicator
2000005	Possible command injection	whoami in HTTP request body	Command-injection detection and correlation
2000006	DNS query	UDP/53 from target	DNS visibility
2000007	Repeated login POST requests	POST + /login.php + per-source threshold	Tuned detection / false-positive reduction

The command-injection detection was initially recorded as SID 2000005 rev2 during the incident evidence. It was later refined to rev3 with HTTP request-body scoping and an explicit priority as part of the tuning phase. Preserving this distinction is important: the incident record reflects the rule version that actually generated the historical alert, while the final active ruleset reflects the later engineering improvement.

```

alert tcp $EXTERNAL_NET any -> $HOME_NET 4280 (
  msg:"PROJECT Possible Command Injection - WHOAMI";
  flow:to_server,established; http_client_body;
  content:"whoami",nocase; priority:1;
  sid:2000005; rev:3;
)

```



```

Sep 19 14:30
assem@assem-VMware-Virtual-Platform: ~
assem@assem-VMware-Virtual-Platform:~$ sudo ls -lh /var/log/snort/alert_fast.txt
[sudo] password for assem:
-rw-r----- 1 root root 3.0K Sep 19 14:43 /var/log/snort/alert_fast.txt
assem@assem-VMware-Virtual-Platform:~$ sudo sed -n '1,20p' /var/log/snort/alert_fast.txt
09/19-14:38:34.957317 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:38:35.958858 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:38:36.960259 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:38:37.961886 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:39:42.296924 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:26.225752 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:27.254319 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:28.275763 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:29.298008 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:35.516208 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:35.516223 ** [1:2000001:1] "PROJECT ICMP Ping Detected" ** [Priority: 0] {ICMP} 192.168.10.10 -> 192.168.20.10
09/19-14:42:36.074082 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.20.10:
09/19-14:42:36.074083 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.
09/19-14:42:36.074084 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.
09/19-14:42:36.074085 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.20.10:
09/19-14:42:36.074410 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.20.10:
09/19-14:42:36.074413 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.20.10:
09/19-14:42:36.075051 ** [1:2000003:1] "PROJECT Possible TCP SYN Scan" ** [Priority: 0] {TCP} 192.168.10.10:53928 -> 192.168.20.10:
assem@assem-VMware-Virtual-Platform:~$

```

Figure 1 — Selected Snort alert log evidence showing custom project detections written to alert_fast.txt.

Lab screenshot captured during the project execution.

9. Packet Analysis with Wireshark

Wireshark was used to validate that the generated traffic matched the intended network behavior. Rather than relying only on IDS output, packet analysis was used to inspect ICMP reachability, TCP handshakes, SYN-scan characteristics, encoded SQL-injection traffic, command-injection requests, repeated HTTP traffic, and DNS queries.

The most valuable packet-analysis evidence was the web-exploitation traffic because it linked application-layer payloads to the later IDS and endpoint findings.

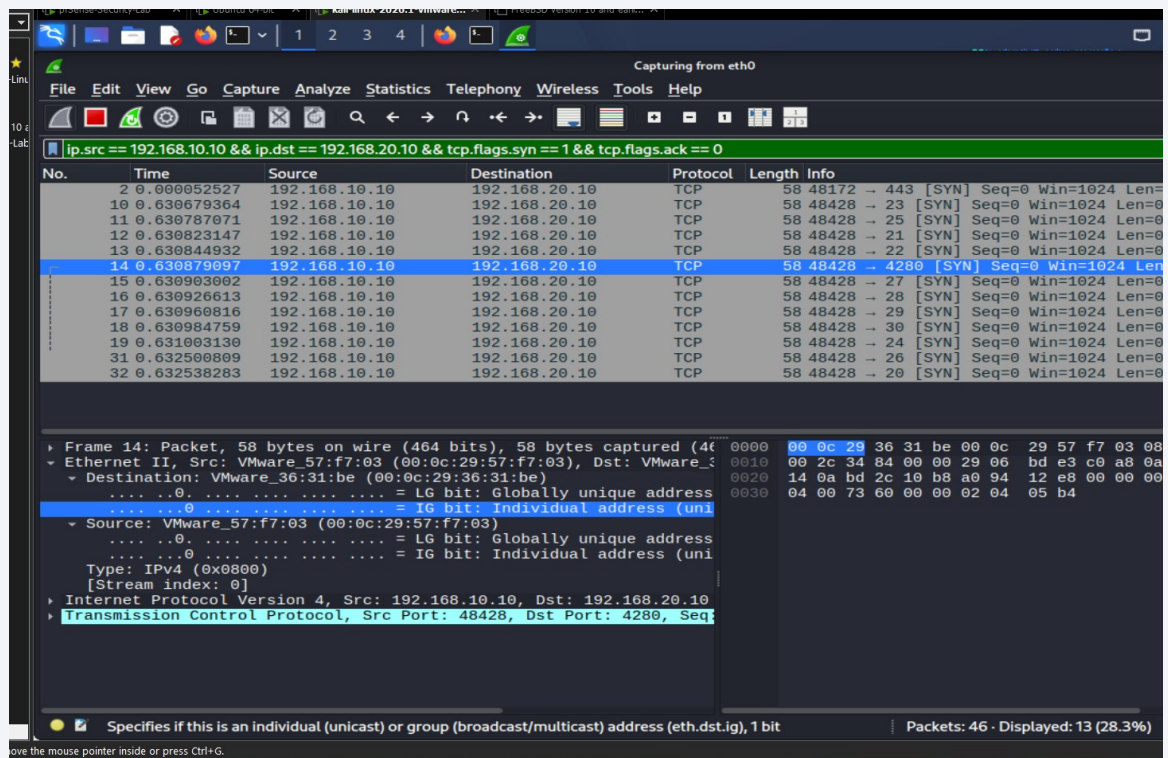


Figure 2 — Wireshark view of SYN-scan traffic generated from Kali toward the target environment.

Lab screenshot captured during the project execution.

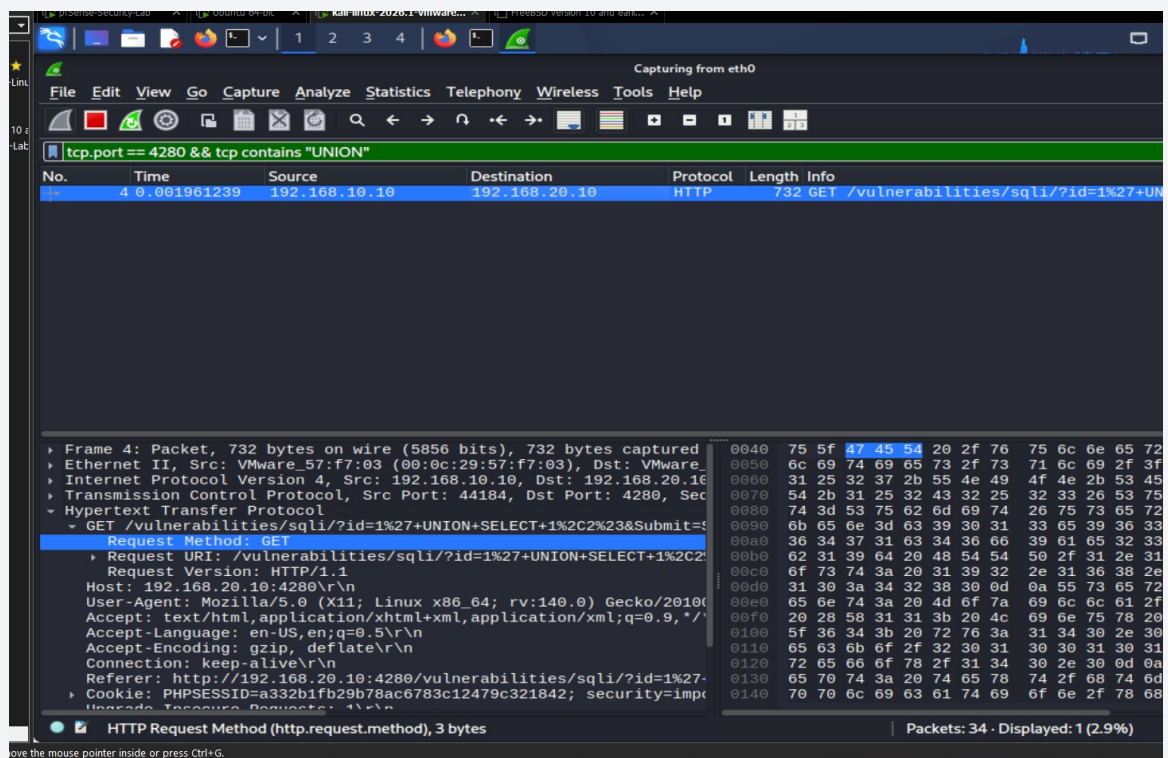


Figure 3 — Wireshark evidence of the controlled UNION SELECT SQL-injection request sent to DVWA.

Lab screenshot captured during the project execution.

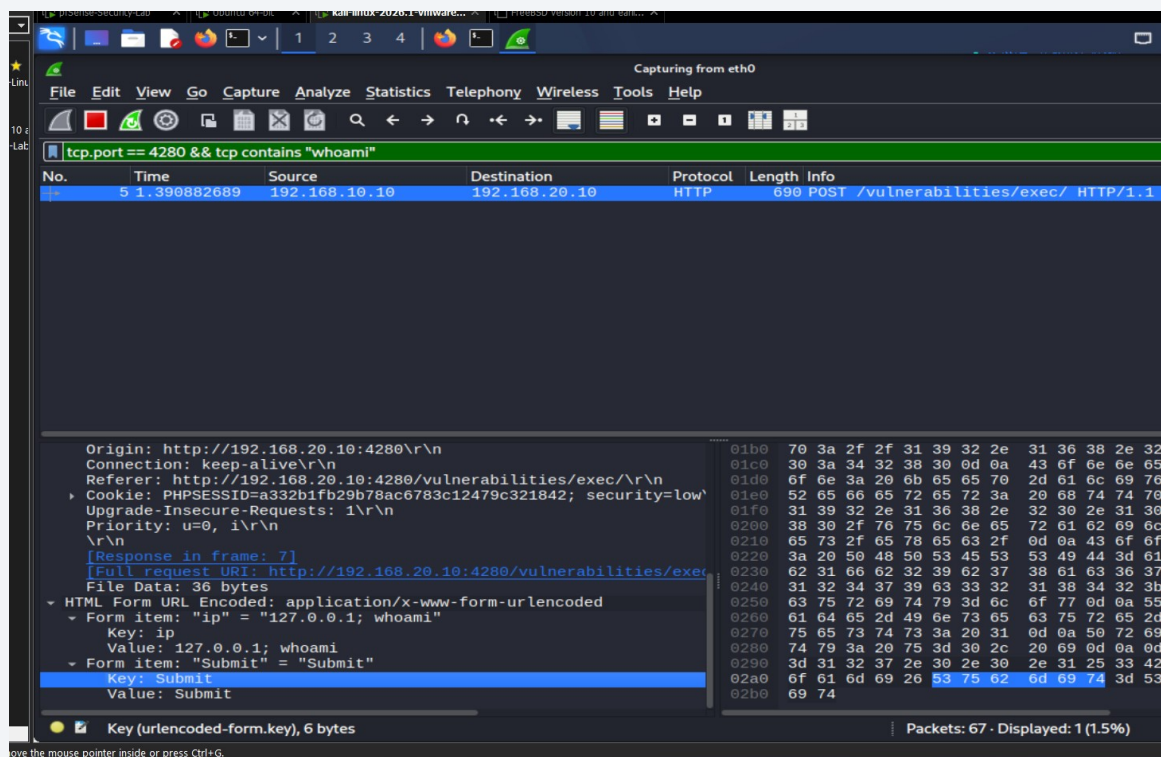


Figure 4 — Wireshark packet evidence containing the controlled whoami command-injection payload.

Lab screenshot captured during the project execution.

10. SIEM Investigation with Splunk

10.1 Log Ingestion and Field Extraction

Snort alert logs were ingested into a dedicated Splunk index using the `snort_alerts` sourcetype. Field extraction converted raw alert strings into analyst-friendly fields such as alert name, protocol, source IP, source port, destination IP, and destination port.

```
index=blue_team sourcetype=snort_alerts
| rex field=_raw "\"(?<alert_name>PROJECT[^\"]+)"\"
| rex field=_raw "{(?<protocol>[A-Z]+)}s+(?<src_ip>\d+\.\d+\.\d+\.\d+)(?::(?<src_port>\d+))?s+-(?<dest_ip>\d+\.\d+\.\d+\.\d+)(?::(?<dest_port>\d+))?"
```

This allowed investigation by source, alert type, and time rather than manually reading raw log lines. The final dashboard included the updated web-exploitation events alongside earlier network detections.

10.2 Investigation Results

Detection	Observed Count
ICMP Ping Detected	12
Possible TCP SYN Scan	9
DVWA TCP Connection Attempt	2
Possible Command Injection - WHOAMI	1
Possible SQL Injection - UNION SELECT	1

The source-IP analysis showed 192.168.10.10 as the source for the 25 summarized events in the final investigation view. The purpose of the dashboard was not only visualization; it demonstrated that multiple detection types could be pivoted around a common source and reviewed as a timeline.



Figure 5 — Splunk SOC dashboard summarizing alert counts, source activity, and detection trends.

Lab screenshot captured during the project execution.

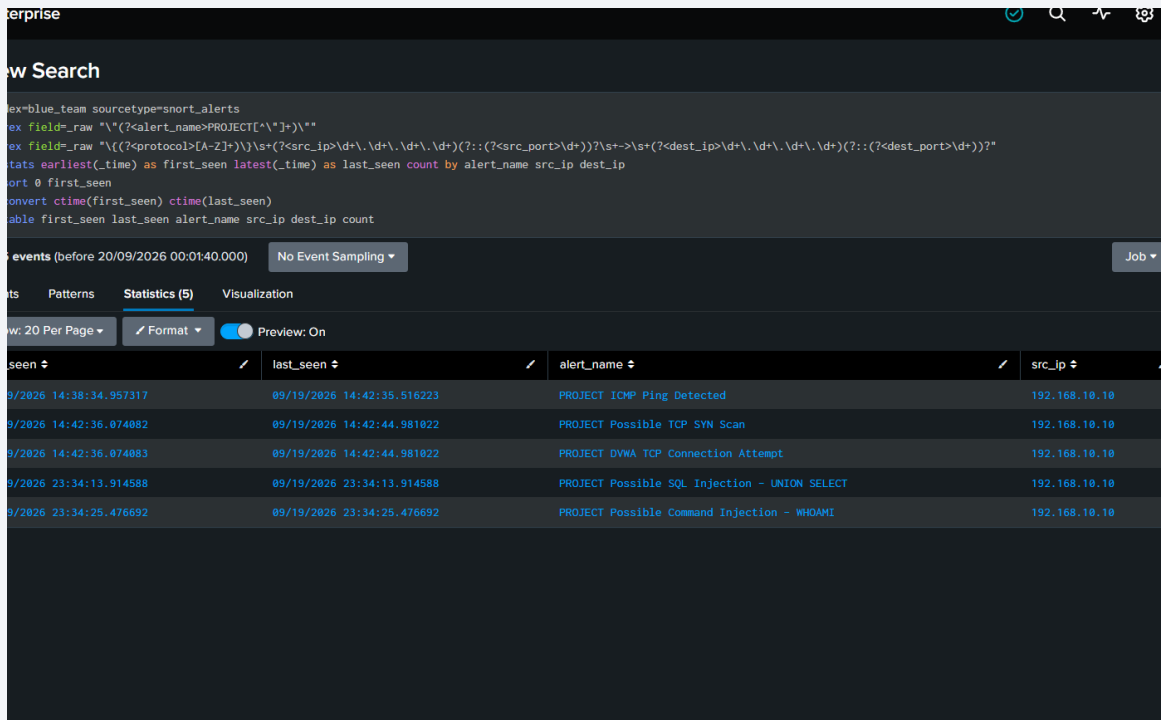


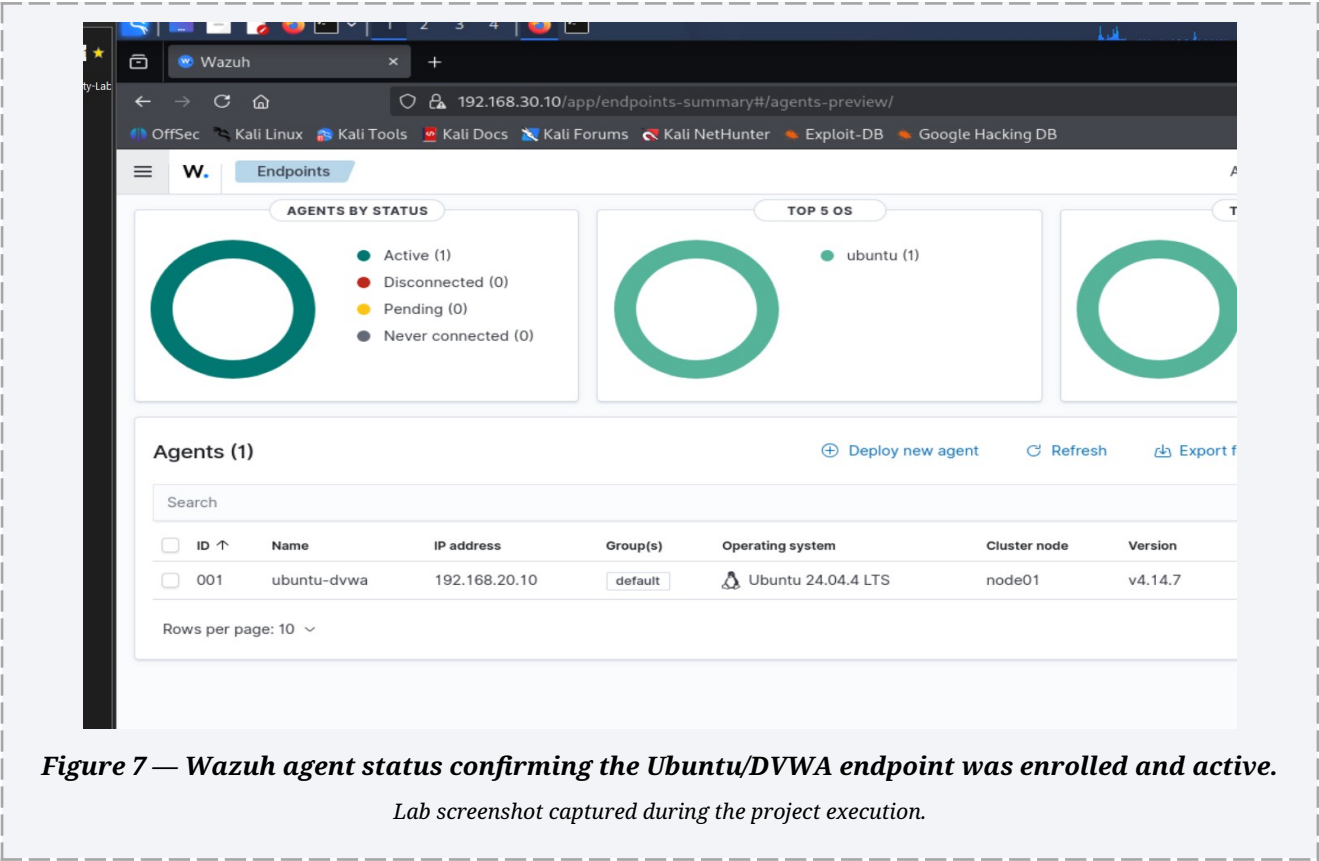
Figure 6 — Splunk detection summary consolidating the principal Snort alert categories used in the investigation.

Lab screenshot captured during the project execution.

11. Endpoint Monitoring with Wazuh and Auditd

11.1 Wazuh Architecture

A Wazuh 4.14.7 appliance was deployed on the SOC network at 192.168.30.10. The Ubuntu target used a Wazuh agent named ubuntu-dvwa. Connectivity from the DMZ to the SOC network was explicitly enabled where required for agent communications, while the monitoring server remained logically separated from the target and attacker segments.



11.2 Auditd Integration

Auditd was configured to record execve activity for the www-data execution context used by the web application. The audit key audit-wazuh-c allowed relevant process-execution events to be searched quickly and fed into Wazuh. The rules were made persistent under /etc/audit/rules.d so that the monitoring survived target reboots.

This endpoint visibility became the decisive evidence for the command-injection incident because it showed not only that a suspicious request crossed the network, but that shell and utility processes actually executed on the target.

11.3 File Integrity Monitoring

Wazuh File Integrity Monitoring was configured for real-time monitoring of /etc with Whodata support. The controlled persistence scenario used /etc/cron.d because cron is a meaningful Linux persistence location and because file lifecycle events could be observed without introducing malicious functionality.

12. Incident Case Study 1 - DVWA Command Injection

12.1 Scenario

Kali (192.168.10.10) sent a controlled command-injection request to the DVWA service on Ubuntu (192.168.20.10:4280). The submitted payload caused the application to execute whoami in the context of the web-service account. The exercise was designed to test whether the SOC stack could progress from network detection to endpoint confirmation.

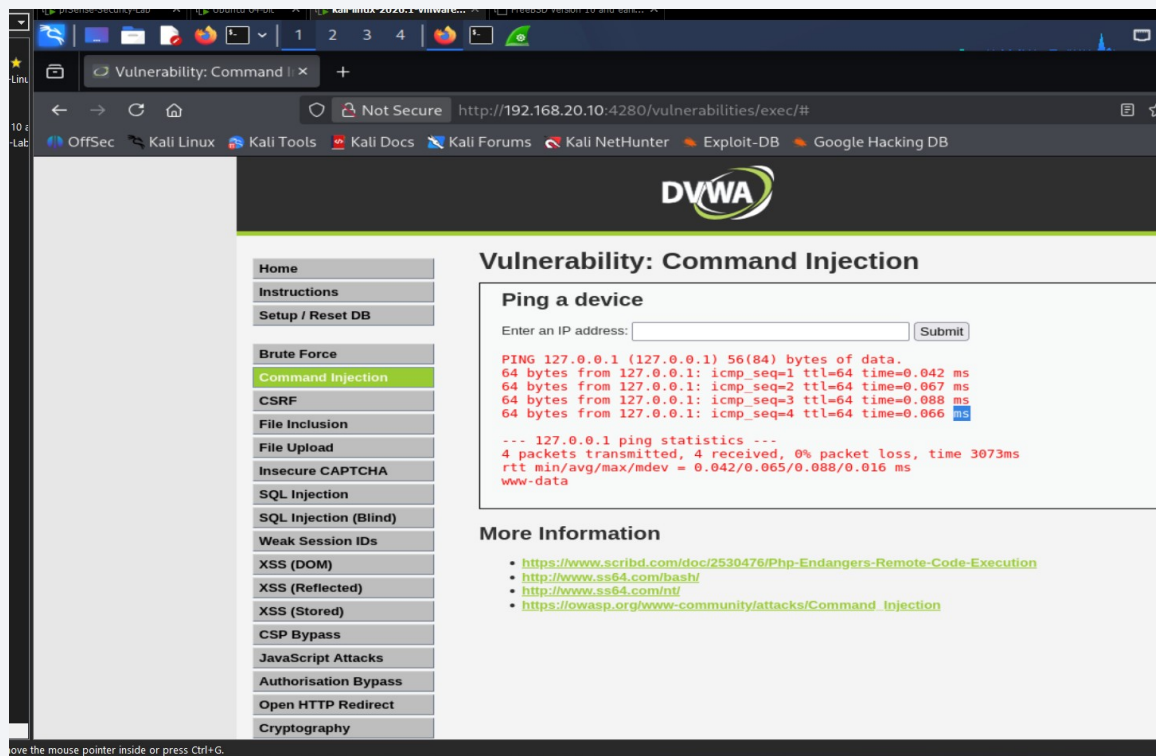


Figure 8 — DVWA response showing the controlled command-injection execution result under the web-service context.

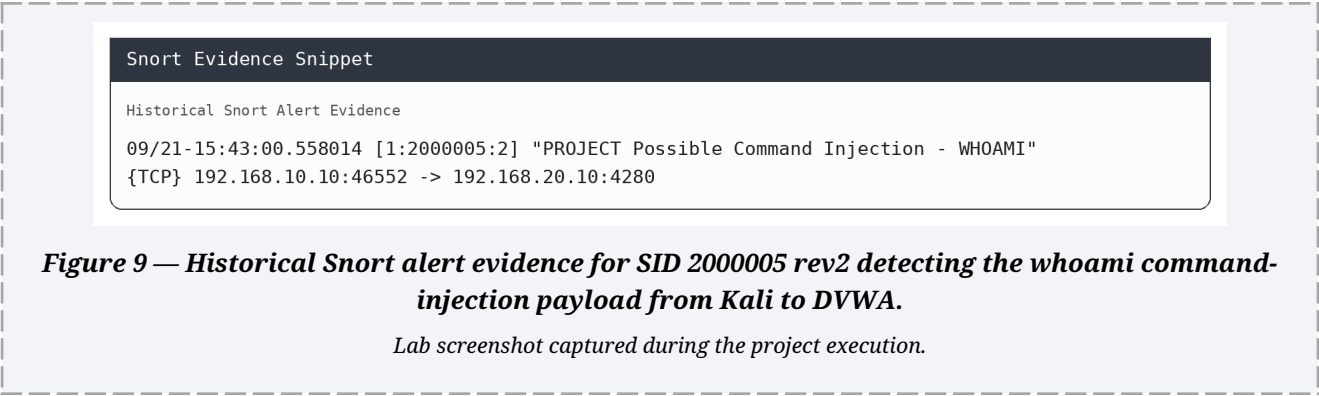
Lab screenshot captured during the project execution.

12.2 Network Detection

Snort generated the following historical alert during the incident window:

```
09/21-15:43:00.558014 [1:2000005:2]
"PROJECT Possible Command Injection - WHOAMI"
{TCP} 192.168.10.10:46552 -> 192.168.20.10:4280
```

The alert established that the whoami indicator was observed in traffic from Kali to the DVWA service. At this point, the correct analyst conclusion was a suspicious command-injection attempt - not yet proof that the operating system executed the command.



12.3 Endpoint Evidence

Auditd then recorded a clear execution chain under www-data in the Docker application context. The timestamps closely followed the Snort event, providing independent endpoint evidence.

Time	Endpoint Evidence	Interpretation
15:43:00.662	/usr/bin/dash executed as www-data	The vulnerable application spawned a Unix shell.
15:43:00.673	/usr/bin/ping executed as www-data	The command chain began executing on the endpoint.
15:43:03.780	/usr/bin/whoami executed as www-data	Successful command execution was confirmed.

The Wazuh manager produced rule 80792 alerts from this Auditd telemetry. The event details included the ubuntu-dvwa agent, command fields, the executable path, and EUID 33 (www-data).

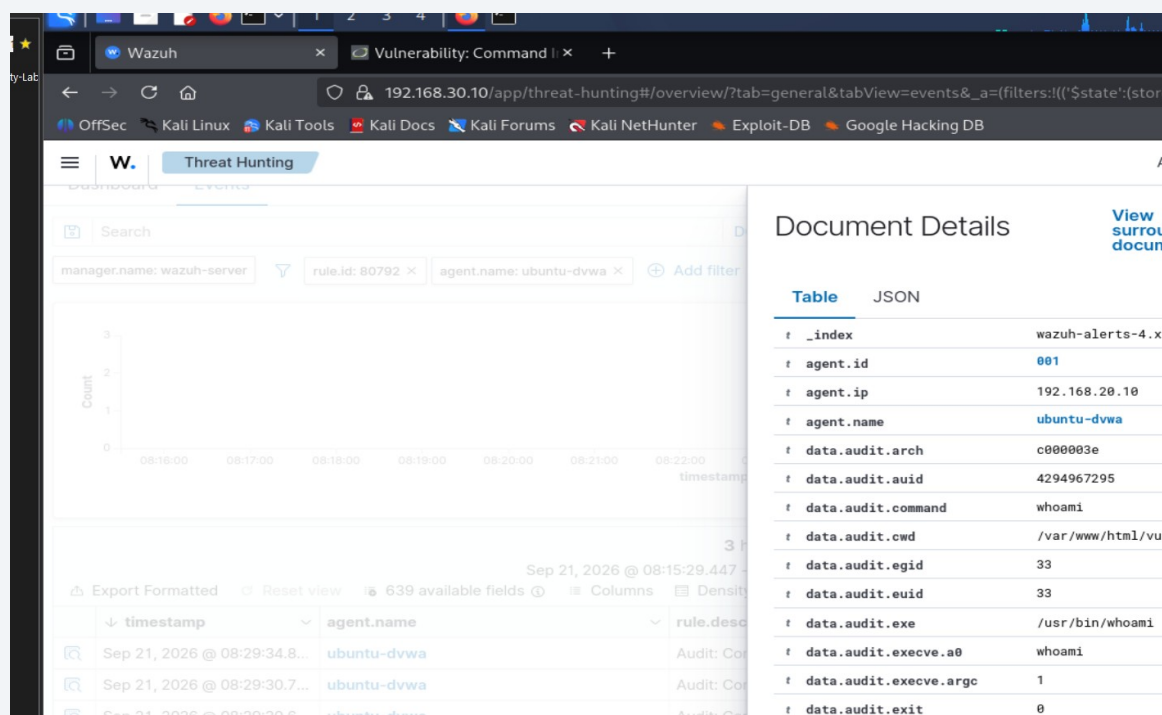


Figure 10 — Wazuh endpoint telemetry for the DVWA command-injection incident, showing the whoami execution event on the ubuntu-dvwa host.

Lab screenshot captured during the project execution.

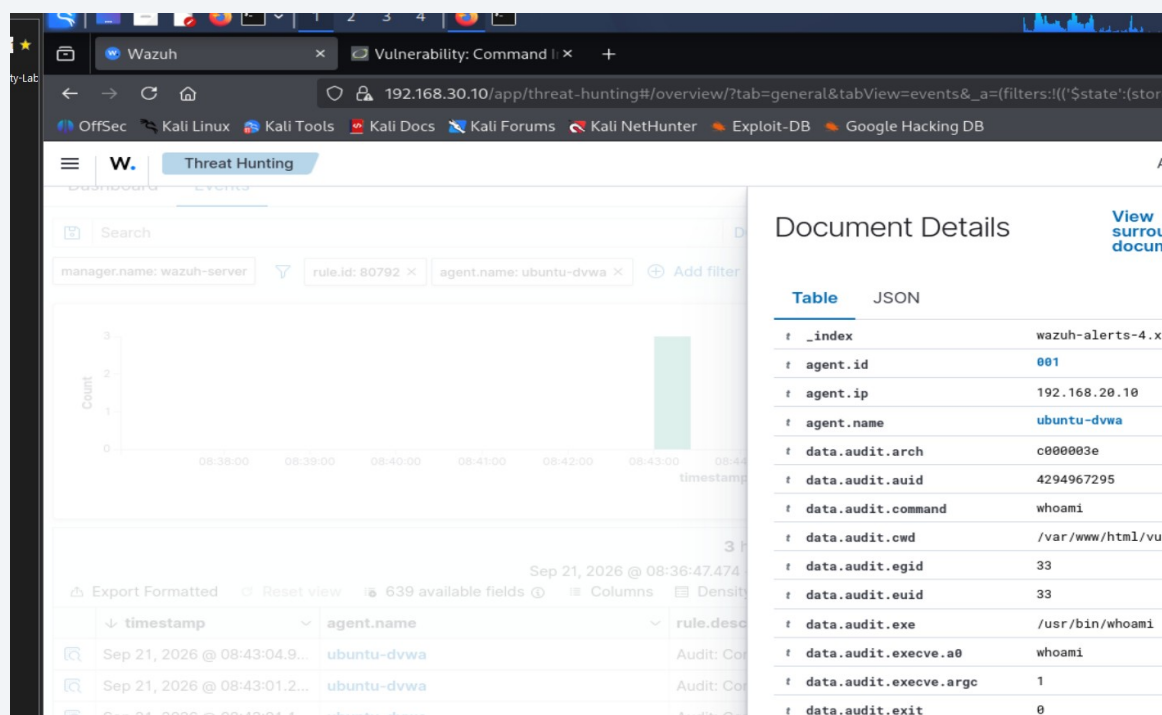


Figure 11 — Detailed Wazuh/Auditd evidence for the whoami process executed under the www-data context.

Lab screenshot captured during the project execution.

12.4 Network-to-Endpoint Correlation

Snort detected the payload at 15:43:00.558. Auditd recorded /usr/bin/dash at 15:43:00.662. The difference was approximately 0.104 seconds. The close timestamps, matching command activity, source/target context, and execution identity created a strong correlation between the network request and endpoint behavior.

Analyst conclusion

Snort alone supported the conclusion "suspicious command-injection traffic reached the application." Snort + Auditd + Wazuh supported the stronger conclusion "the controlled request resulted in actual command execution on the target."

12.5 Incident Timeline

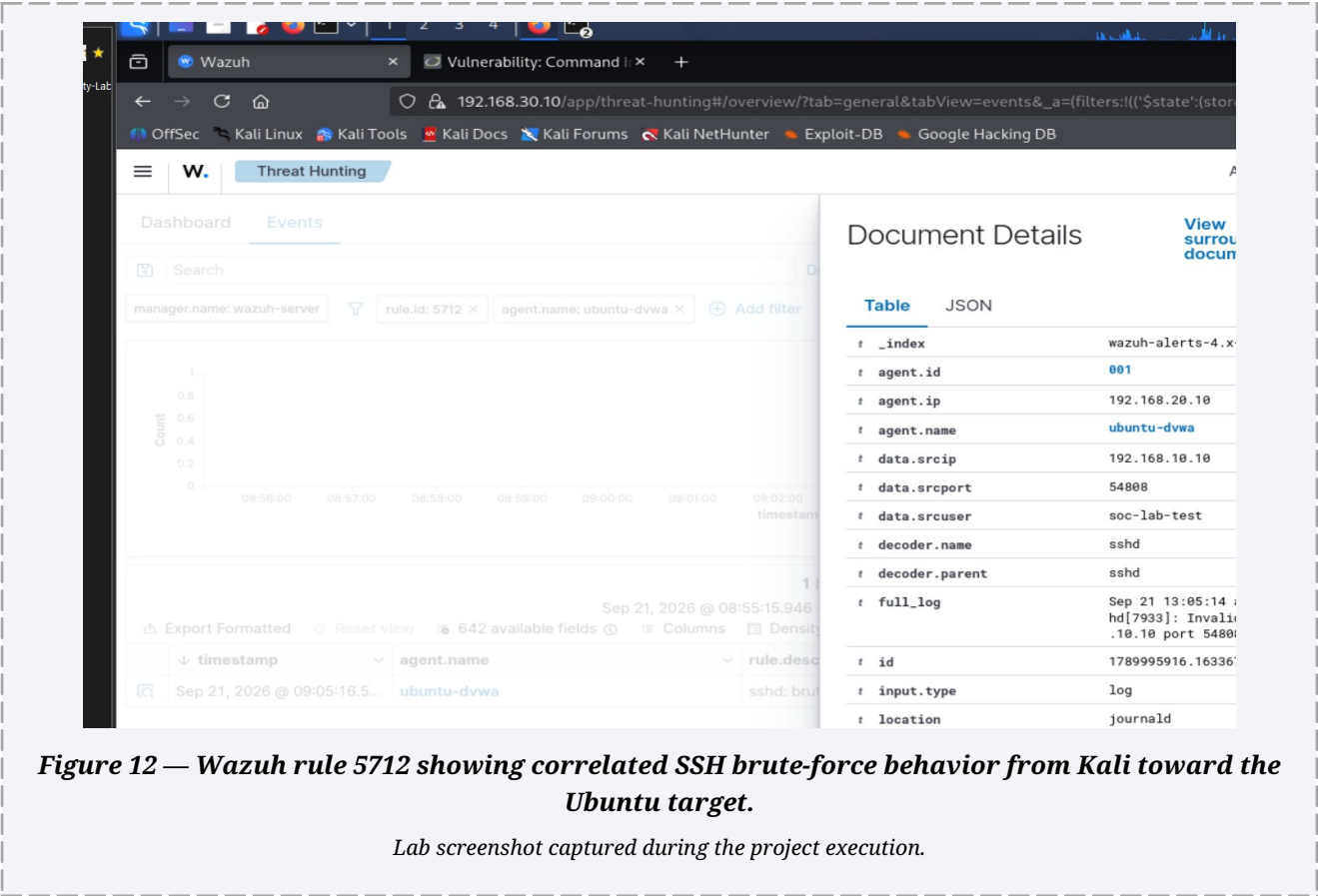
Time	Source	Event / Interpretation
15:43:00.558	Snort	SID 2000005 rev2 observed WHOAMI payload from 192.168.10.10:46552 to 192.168.20.10:4280.
15:43:00.662	Auditd	/usr/bin/dash executed as www-data; shell execution began.
15:43:00.673	Auditd	/usr/bin/ping executed as www-data.
15:43:03.780	Auditd	/usr/bin/whoami executed as www-data; endpoint execution confirmed.
Same incident window	Wazuh	Rule 80792 surfaced the endpoint command-execution telemetry.
Response phase	pfSense	Targeted block applied from 192.168.10.10 to 192.168.20.10.
Recovery phase	pfSense	Containment rule disabled after validation; connectivity restored.

13. Incident Case Study 2 - SSH Authentication Attack / Brute Force

13.1 Controlled Scenario

OpenSSH Server was enabled on the Ubuntu target and verified on TCP/22. From Kali, repeated incorrect passwords were submitted against the deliberately nonexistent lab username soc-lab-test. The goal was detection validation, not account compromise.

Ubuntu authentication logs showed the source 192.168.10.10 and the invalid username. Wazuh detected the authentication failures and generated the higher-level brute-force alert rule 5712 after repeated activity from the same source.



Scope statement

No valid account was compromised. This was a controlled password-guessing / brute-force detection exercise intended to validate authentication monitoring and event correlation.

13.2 Analyst Interpretation

A single failed login is usually insufficient to classify activity as brute force. The meaningful signal came from repetition, common source context, the same target service, and Wazuh correlation. The scenario reinforced the distinction between individual authentication failures and a pattern that merits escalation.

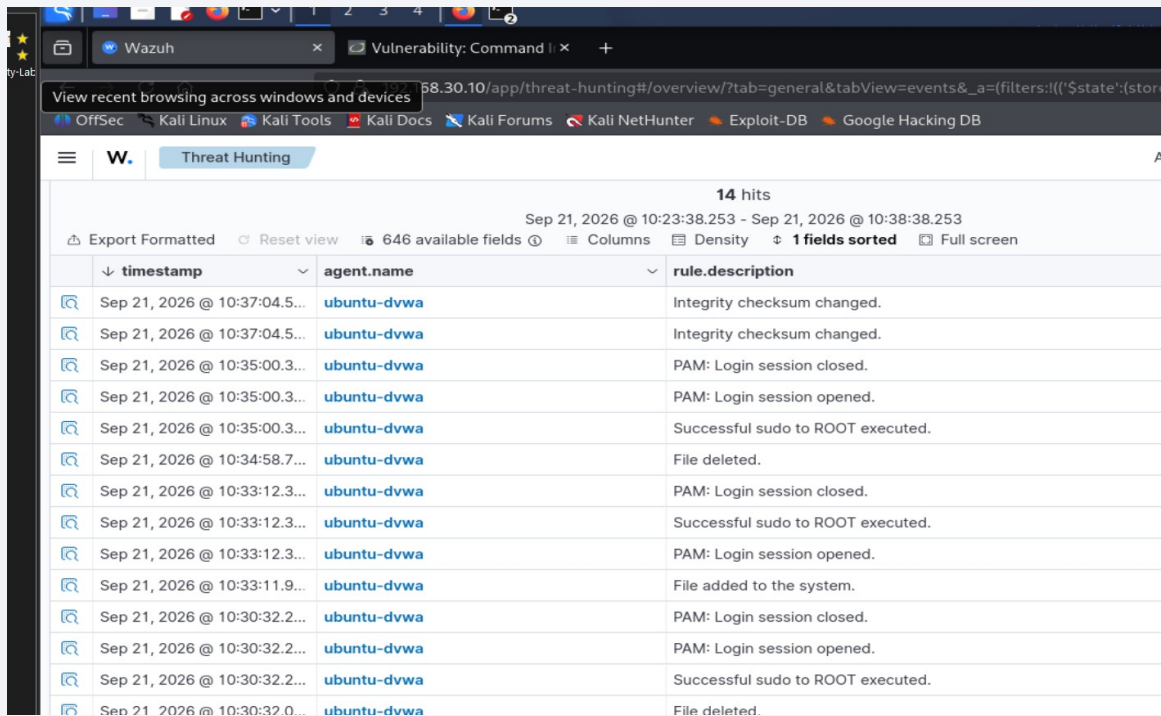
14. Incident Case Study 3 - File Integrity and Persistence Monitoring

14.1 Persistence Simulation

A harmless cron entry under /etc/cron.d was used to simulate a persistence-relevant file change. The payload performed no malicious action; its purpose was to test whether Wazuh FIM could observe file lifecycle changes in a security-sensitive path.

Lifecycle Stage	Wazuh Rule	Observed Meaning
Create	554	File added to the system
Modify	550	Integrity checksum changed
Delete	553	File deleted

Whodata monitoring also provided identity-related context for the change. The strongest evidence was the combined create -> modify -> delete sequence rather than any single FIM event.



The screenshot shows the Wazuh Threat Hunting interface. The top navigation bar includes 'Wazuh' and 'Threat Hunting'. Below the navigation bar, there's a search bar and a table of events. The table has columns for 'timestamp', 'agent.name', and 'rule.description'. The events are filtered by 'Sep 21, 2026 @ 10:23:38.253 - Sep 21, 2026 @ 10:38:38.253'. The table shows 14 hits. The events are as follows:

timestamp	agent.name	rule.description
Sep 21, 2026 @ 10:37:04.5...	ubuntu-dvwa	Integrity checksum changed.
Sep 21, 2026 @ 10:37:04.5...	ubuntu-dvwa	Integrity checksum changed.
Sep 21, 2026 @ 10:35:00.3...	ubuntu-dvwa	PAM: Login session closed.
Sep 21, 2026 @ 10:35:00.3...	ubuntu-dvwa	PAM: Login session opened.
Sep 21, 2026 @ 10:35:00.3...	ubuntu-dvwa	Successful sudo to ROOT executed.
Sep 21, 2026 @ 10:34:58.7...	ubuntu-dvwa	File deleted.
Sep 21, 2026 @ 10:33:12.3...	ubuntu-dvwa	PAM: Login session closed.
Sep 21, 2026 @ 10:33:12.3...	ubuntu-dvwa	Successful sudo to ROOT executed.
Sep 21, 2026 @ 10:33:12.3...	ubuntu-dvwa	PAM: Login session opened.
Sep 21, 2026 @ 10:33:11.9...	ubuntu-dvwa	File added to the system.
Sep 21, 2026 @ 10:30:32.2...	ubuntu-dvwa	PAM: Login session closed.
Sep 21, 2026 @ 10:30:32.2...	ubuntu-dvwa	PAM: Login session opened.
Sep 21, 2026 @ 10:30:32.2...	ubuntu-dvwa	Successful sudo to ROOT executed.
Sep 21, 2026 @ 10:30:32.0...	ubuntu-dvwa	File deleted.

Figure 13 — Wazuh FIM summary showing the controlled cron file lifecycle: added, modified, and deleted.

Lab screenshot captured during the project execution.

Scope statement

The cron file was a controlled persistence simulation. It did not install malware or establish unauthorized persistence outside the local lab.

15. Incident Response and Containment

15.1 Response Logic

After the suspicious source had been validated through the controlled incidents, pfSense was used to demonstrate targeted network containment. The response workflow followed a simple SOC sequence: detection -> validation -> containment -> verification -> recovery.

Because Kali-to-DMZ traffic entered pfSense through LAN, the containment rule was placed on the LAN interface. The rule blocked source 192.168.10.10 from destination 192.168.20.10 and was placed above the broad LAN allow rule so that first-match evaluation would apply the block first.

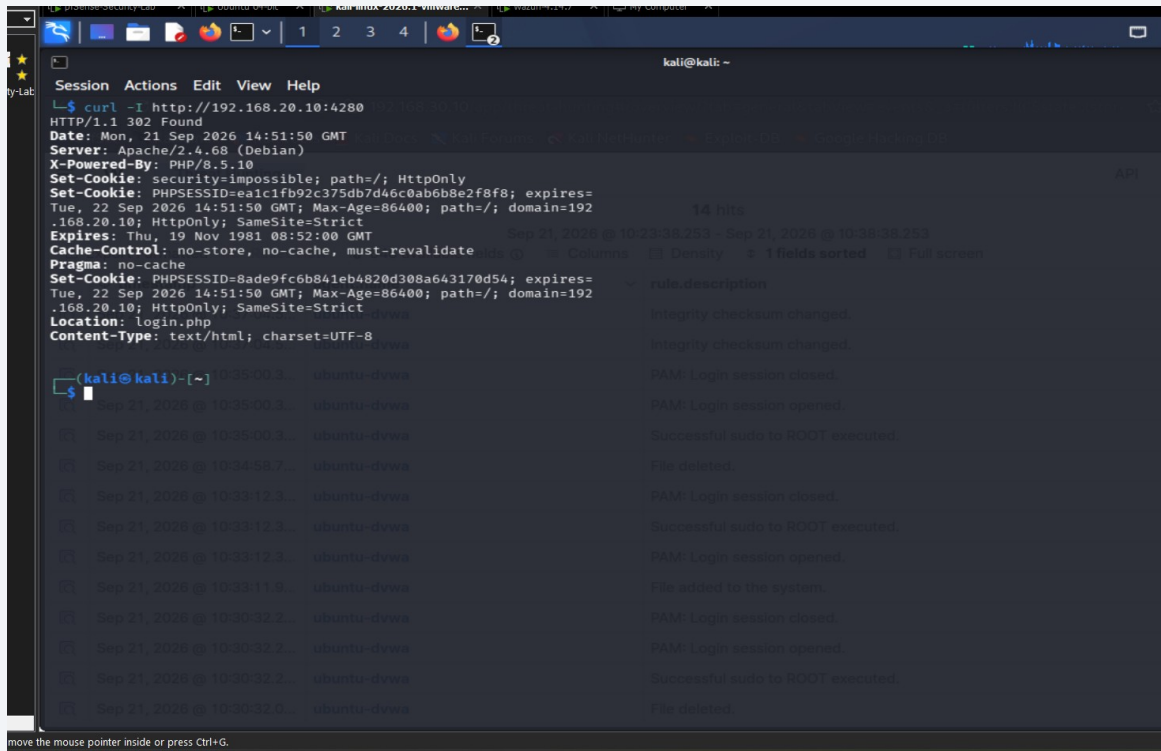


Figure 14 — Pre-containment baseline confirming connectivity from Kali to the Ubuntu/DVWA target before the response action.

Lab screenshot captured during the project execution.

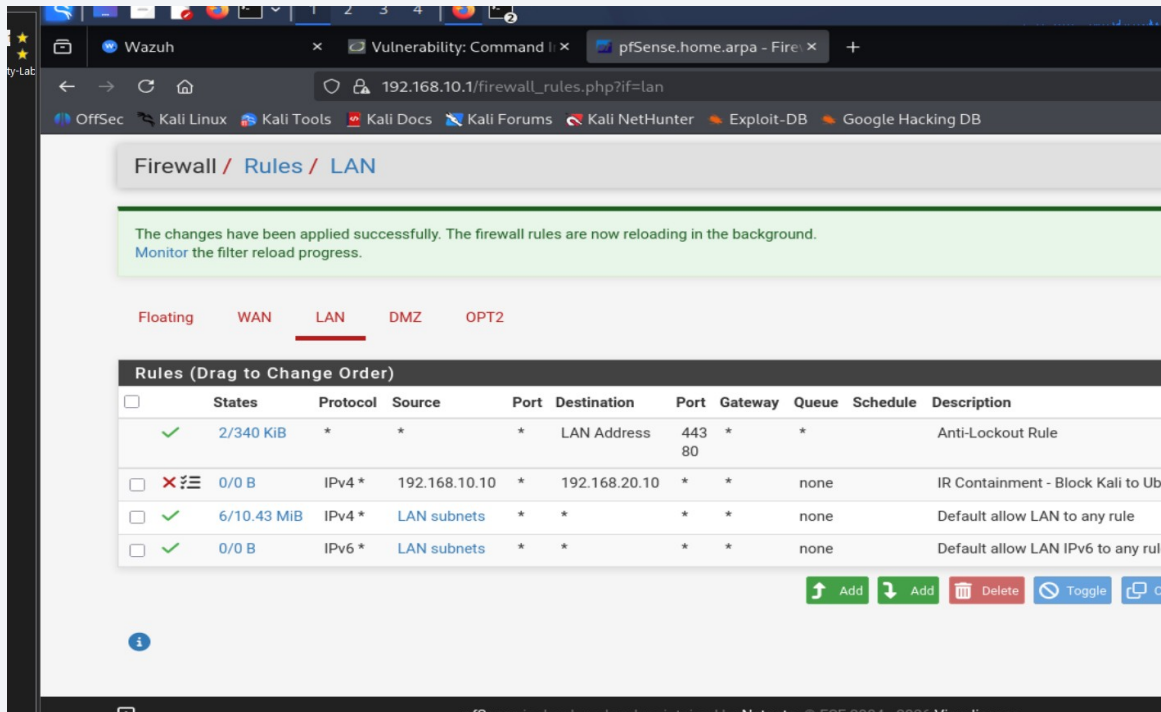


Figure 15 — pfSense LAN containment rule blocking the confirmed suspicious source from the Ubuntu/DVWA target.

Lab screenshot captured during the project execution.

15.2 Verification

Containment was validated from Kali using three different service checks: ICMP reachability, SSH TCP/22, and HTTP access to DVWA TCP/4280. The tests failed after the rule was applied, demonstrating that the response action affected the intended source-to-target path rather than relying on an assumed firewall state.

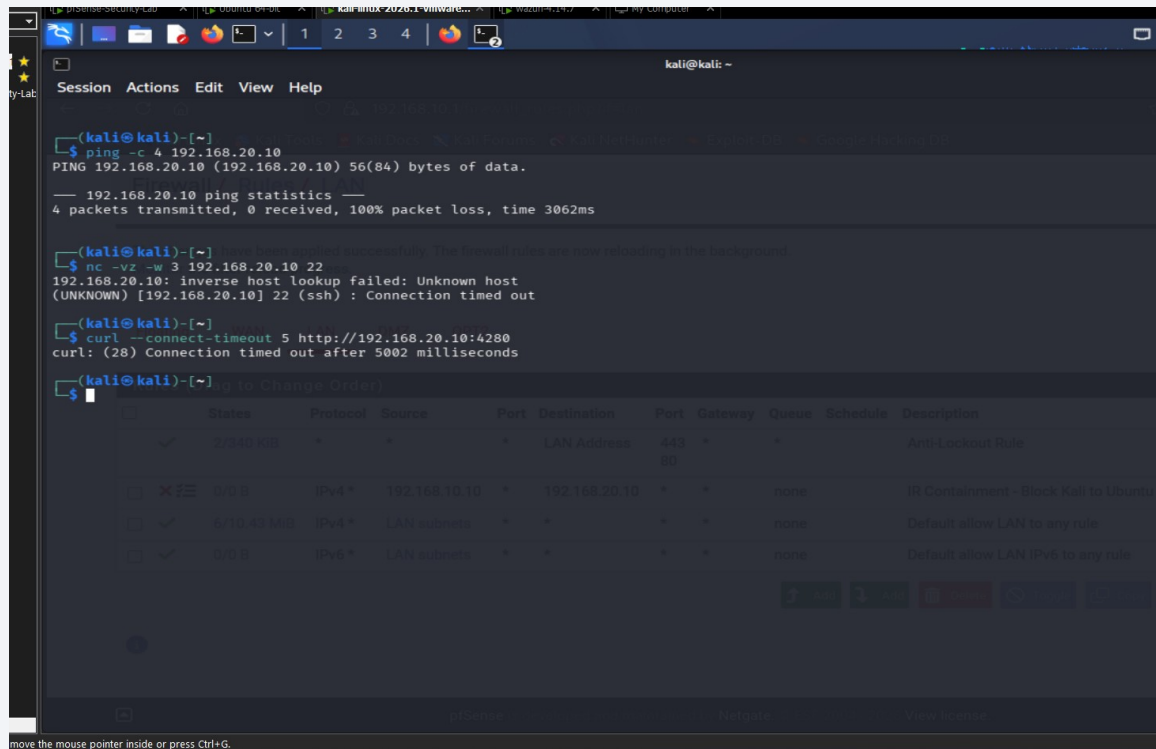


Figure 16— Post-containment verification showing loss of direct connectivity from Kali to the Ubuntu/DVWA target.

Lab screenshot captured during the project execution.

pfSense firewall logs were then reviewed to verify that the firewall - rather than an application failure - was responsible for the blocked traffic.

The screenshot shows the pfSense firewall log interface. The browser address bar indicates the URL is 192.168.10.1/status_logs_filter.php. The log table contains the following data:

Time	Interface	Source	Destination	Protocol	Port	Action
Sep 21 23:06:00	LAN	(@4294967295)				
Sep 21 23:06:00	LAN	(@4294967295)				
Sep 21 23:06:01	DMZ	(@4294967295)				
Sep 21 23:06:01	DMZ	(@4294967295)				
Sep 21 23:06:02	OPT2	(@4294967295)				
Sep 21 23:06:02	OPT2	(@4294967295)				
Sep 21 23:12:36	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10			
Sep 21 23:12:37	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10			
Sep 21 23:12:38	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10			
Sep 21 23:12:39	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10			
Sep 21 23:12:56	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10:40424			
Sep 21 23:12:57	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10:40424			
Sep 21 23:12:58	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10:40424			
Sep 21 23:12:59	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10:40424			
Sep 21 23:13:00	LAN	IR Containment - Block Kali to Ubuntu (1790002830)	192.168.10.10:40424			

Figure 17 — pfSense firewall log evidence confirming traffic from 192.168.10.10 to 192.168.20.10 was blocked by the containment policy.

Lab screenshot captured during the project execution.

15.3 Recovery

The containment rule was disabled rather than deleted so the response control remained documented. Connectivity tests were repeated and normal access returned. This demonstrated the operational distinction between containment and recovery: containment restricts the suspected threat path, while recovery restores normal service after the analyst determines it is appropriate to do so.

```

kali@kali: ~
Session Actions Edit View Help
4 packets transmitted, 0 received, 100% packet loss, time 3051ms

(kali@kali)~$ curl --connect-timeout 5 http://192.168.20.10:4280
curl: (28) Connection timed out after 5000 milliseconds

(kali@kali)~$ ping -c 4 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data:
64 bytes from 192.168.20.10: icmp_seq=1 ttl=63 time=0.914 ms
64 bytes from 192.168.20.10: icmp_seq=2 ttl=63 time=0.753 ms
64 bytes from 192.168.20.10: icmp_seq=3 ttl=63 time=0.839 ms
64 bytes from 192.168.20.10: icmp_seq=4 ttl=63 time=0.689 ms

--- 192.168.20.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/mdev = 0.689/0.798/0.914/0.085 ms

(kali@kali)~$ nc -vz -w 3 192.168.20.10 22
192.168.20.10: inverse host lookup failed: Host name lookup fail
ure
(UNKNOWN) [192.168.20.10] 22 (ssh) open

(kali@kali)~$ curl -I --connect-timeout 5 http://192.168.20.10:4280
HTTP/1.1 302 Found
Date: Mon, 21 Sep 2026 20:37:56 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.10
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=be4b2de5083ee195af32fc7edeb8cd49; expires=
Tue, 22 Sep 2026 20:37:56 GMT; Max-Age=86400; path=/; domain=192
.168.20.10; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=3bcfa73a8a39f011844f9252418f9c18; expires=
Tue, 22 Sep 2026 20:37:56 GMT; Max-Age=86400; path=/; domain=192
.168.20.10; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8

```

Figure 18 — Post-recovery connectivity confirming normal access returned after the containment rule was disabled.

Lab screenshot captured during the project execution.

16. Detection Engineering and False-Positive Reduction

16.1 Original Detection Problem

The original SID 2000007 logic alerted on repeated DVWA traffic from the Kali source to TCP/4280. While useful for early validation, it could classify rapid but benign requests as suspicious because it did not distinguish HTTP method or resource context. This made the rule a good candidate for false-positive reduction.

16.2 Tuned Rule

The rule was refined to require an HTTP POST request to /login.php, track matches by source, and alert when at least five matching requests occurred within 30 seconds. The revision was incremented to rev3 and an explicit priority was assigned.

```

alert tcp $EXTERNAL_NET any -> $HOME_NET 4280 (
  msg:"PROJECT Repeated DVWA Login POST Requests";
  flow:to_server,established;
  http_method; content:"POST";
  http_uri; content:"/login.php",nocase;
  detection_filter:track by_src,count 5,seconds 30;
  priority:2; sid:2000007; rev:3;
)

```

16.3 Validation Methodology

The tuned logic was tested against both benign and suspicious traffic. Ten repeated GET requests to /login.php produced no SID 2000007 alert. Repeated POST login requests produced multiple SID 2000007 rev3 alerts. This validated that the rule retained the intended detection while reducing a known benign match condition.

Test	Traffic	Expected Result	Observed Result
Benign validation	10 repeated GET /login.php requests	No SID 2000007 alert	No alert
Suspicious validation	Repeated POST /login.php requests	SID 2000007 rev3 alerts	Multiple alerts

```

Sep 22 00:28
assem@assem-VMware-Virtual-Platform:~/DVWA
09/22-00:17:52.959997 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35308 -> 19
09/22-00:17:52.972686 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35324 -> 19
09/22-00:17:52.984904 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35334 -> 19
09/22-00:17:52.998102 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35344 -> 19
09/22-00:17:53.010680 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35356 -> 19
09/22-00:17:53.024030 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35372 -> 19
09/22-00:17:53.038034 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35388 -> 19
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo rm -rf /tmp/snort_get_test
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo mkdir /tmp/snort_get_test
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo snort -q \
-c /usr/local/etc/snort/snort.lua \
-r /tmp/dvwa_get_burst.pcap \
-A alert_fast \
-l /tmp/snort_get_test \
-k none \
--lua "alert_fast = { file = true }"
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo ls -lh /tmp/snort_get_test/
total 4.0K
-rw-r----- 1 root root 2.2K Sep 22 00:25 alert_fast.txt
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo grep "2000007" /tmp/snort_get_test/alert_fast.txt
assem@assem-VMware-Virtual-Platform:~/DVWA$ sudo grep "2000007" /tmp/snort_tuned/alert_fast.txt
09/22-00:17:48.856264 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34866 -> 19
09/22-00:17:48.871440 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34872 -> 19
09/22-00:17:48.885133 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34884 -> 19
09/22-00:17:48.902317 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34890 -> 19
09/22-00:17:48.916040 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34900 -> 19
09/22-00:17:52.917323 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35278 -> 19
09/22-00:17:52.933159 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35284 -> 19
09/22-00:17:52.945652 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35294 -> 19
09/22-00:17:52.959997 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35308 -> 19
09/22-00:17:52.972686 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35324 -> 19
09/22-00:17:52.984904 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35334 -> 19
09/22-00:17:52.998102 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35344 -> 19
09/22-00:17:53.010680 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35356 -> 19
09/22-00:17:53.024030 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35372 -> 19
09/22-00:17:53.038034 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35388 -> 19
assem@assem-VMware-Virtual-Platform:~/DVWA$

```

Figure 19 — Tuned SID 2000007 rev3 generating alerts for repeated suspicious POST requests to /login.php.

Lab screenshot captured during the project execution.

```

total 4.0K
-rw-r--r-- 1 root root 2.2K Sep 22 00:25 alert_fast.txt
assem@assem-Virtual-Platform:~/DVWA$ sudo grep "2000007" /tmp/snort_get_test/alert_fast.txt
assem@assem-Virtual-Platform:~/DVWA$ sudo grep "2000007" /tmp/snort_tuned/alert_fast.txt
09/22-00:17:48.856264 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34866 -> 19
09/22-00:17:48.871440 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34872 -> 19
09/22-00:17:48.885133 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34884 -> 19
09/22-00:17:48.902317 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34890 -> 19
09/22-00:17:48.916040 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:34900 -> 19
09/22-00:17:52.917323 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35278 -> 19
09/22-00:17:52.933159 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35284 -> 19
09/22-00:17:52.945652 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35294 -> 19
09/22-00:17:52.959997 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35308 -> 19
09/22-00:17:52.972686 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35324 -> 19
09/22-00:17:52.984904 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35334 -> 19
09/22-00:17:52.998102 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35344 -> 19
09/22-00:17:53.010680 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35356 -> 19
09/22-00:17:53.024030 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35372 -> 19
09/22-00:17:53.038034 [**] [1:2000007:3] "PROJECT Repeated DVWA Login POST Requests" [**] [Priority: 2] (TCP) 192.168.10.10:35388 -> 19
assem@assem-Virtual-Platform:~/DVWA$ sudo nano /user/local/etc/snort/rules/local.rules
sudo mkdir /tmp/snort_get_test
sudo snort -q \
-c /usr/local/etc/snort/snort.lua \
-r /tmp/dvwa_get_burst.pcap \
-A alert_fast \
-l /tmp/snort_get_test \
-k none \
--lua "alert_fast = { file = true }"
assem@assem-Virtual-Platform:~/DVWA$ if sudo grep -q "2000007" /tmp/snort_get_test/alert_fast.txt; then
echo "FAIL: SID 2000007 triggered on benign GET traffic"
else
echo "PASS: No SID 2000007 alert for repeated benign GET requests"
fi
PASS: No SID 2000007 alert for repeated benign GET requests
assem@assem-Virtual-Platform:~/DVWA$

```

Figure 20 — Benign validation showing repeated GET requests did not trigger SID 2000007 after tuning.

Lab screenshot captured during the project execution.

16.4 Troubleshooting the Detection

During validation, tcpdump confirmed that POST /login.php traffic was reaching ens33, but a simplified live Snort test rule did not initially alert. The troubleshooting process reduced the problem systematically: verify packet presence, simplify the signature, capture a PCAP, then replay the PCAP through Snort offline.

The diagnostic rule SID 2000099 successfully alerted during offline PCAP replay for 192.168.10.10:50742 -> 192.168.20.10:4280. This demonstrated that HTTP method/URI signature logic was correct and that the earlier issue was related to live inspection/capture state or timing rather than basic rule syntax. SID 2000099 was a temporary diagnostic rule and was not retained as a production detection.

A similar improvement was applied to SID 2000005 after the historical command-injection alert displayed Priority 0. The final rule was scoped to the HTTP request body and assigned an explicit priority. This improvement did not rewrite the historical evidence; the incident remained documented with the rev2 alert that actually fired at the time.

Detection-engineering principle

A useful tuning cycle is: reproduce -> verify packet presence -> simplify -> isolate -> replay -> restore conditions one by one -> validate against both benign and suspicious traffic.

17. MITRE ATT&CK Mapping

The ATT&CK mappings below describe observed or simulated behaviors in the lab. They are used to categorize techniques, not to prove malicious intent by themselves. The project intentionally limits the mapping to behaviors directly supported by the exercises.

Technique	Name	Lab Mapping	Evidence
T1059.004	Command and Scripting Interpreter: Unix Shell	Execution	/usr/bin/dash followed by ping and whoami under www-data
T1110.001	Brute Force: Password Guessing	Credential Access	Repeated incorrect SSH passwords; Wazuh rule 5712
T1053.003	Scheduled Task/Job: Cron	Persistence / Execution	Controlled file activity under /etc/cron.d; Wazuh FIM create/modify/delete
T1190	Exploit Public-Facing Application	Conceptual Initial Access mapping	DVWA exploitation scenario; lab service was isolated in a DMZ and not actually Internet-facing

T1190 is presented as a conceptual mapping only. The DVWA instance was not publicly exposed to the Internet; it was an intentionally vulnerable application inside an isolated DMZ. The mapping reflects the simulated behavior class rather than a claim about real external exposure.

18. Troubleshooting Highlights

Selected troubleshooting examples are included because they demonstrate analytical method rather than installation history. Each issue was approached by narrowing the problem to a specific layer and validating assumptions with direct evidence.

Problem	Investigation / Root Cause	Fix	Lesson
ICMP from Wazuh network failed while pfSense could reach the host	OPT2 allow rule was configured for TCP only; ping uses ICMP	Changed the temporary outbound rule to protocol Any	Match firewall protocol to the validation traffic; TCP and ICMP are different protocol families
Wazuh alerts existed locally but were not visible in the dashboard	Filebeat test output returned HTTP 401 after a failed password-change attempt left credential state inconsistent	Restored the correct indexer credentials in Filebeat keystore and restarted Filebeat	Validate every stage of the data path: agent -> manager -> shipper -> indexer -> dashboard
Audit rule needed to survive reboot	Runtime auditctl rules alone were insufficient for reliable restart behavior	Stored rules under /etc/audit/rules.d and loaded them through augenrules	Security telemetry must be persistent, not only correct in the current shell session
Live Snort did not alert on a simplified POST rule although packets were present	tcpdump proved traffic reached ens33; offline PCAP replay triggered SID 2000099	Used capture/replay to isolate live-inspection state from signature logic	Packet visibility, rule matching, and live engine state are separate troubleshooting layers

19. Key Security Findings

Finding	Evidence	Analyst Significance
Command injection produced real endpoint execution	Snort SID 2000005 + Auditd dash/ping/whoami + Wazuh rule 80792	Network and endpoint evidence together converted a suspicious request into a confirmed controlled incident
SSH brute force was observable as a pattern	Authentication logs + Wazuh rule 5712	Repeated failures provide stronger context than one invalid login
Persistence-relevant file changes were visible	Wazuh FIM rules 554, 550, 553 on /etc/cron.d	FIM can surface security-relevant changes even without network indicators
Targeted containment worked without shutting down the target	pfSense source-to-target block + verification + firewall logs	Response can isolate a suspicious path while preserving wider service availability
Detection tuning reduced benign noise	GET produced no 2000007 alert; POST burst produced multiple rev3 alerts	Rules should be validated against both negative and positive test cases

20. Skills Demonstrated

Skill Area	Demonstrated Work
Network Security	pfSense interface design, routing, stateful filtering, segmentation, rule ordering, targeted containment
Network Detection	Custom Snort 3 rules, rule validation, thresholds, HTTP sticky buffers, PCAP replay
Packet Analysis	Wireshark inspection of handshakes, scans, SQLi, command injection, DNS, repeated HTTP traffic
SIEM / SPL	Snort log ingestion, regex field extraction, statistics, source analysis, timelines, dashboarding
Endpoint Monitoring	Wazuh agent deployment, Auditd execve monitoring, FIM with Whodata
Incident Investigation	Timeline reconstruction, cross-source correlation, attempt-vs-execution validation
Incident Response	Containment, verification, firewall log review, recovery
Detection Engineering	False-positive analysis, tuned POST rule, benign/suspicious validation, troubleshooting methodology
Threat Frameworks	Conservative MITRE ATT&CK mapping tied to observed lab behaviors

21. Lessons Learned

- An IDS alert is evidence of suspicious traffic, not automatic proof of successful compromise.

- Cross-layer correlation substantially increases confidence: network payloads, endpoint processes, authentication logs, file changes, and firewall actions answer different parts of the incident story.
- Firewall rule placement depends on traffic direction and ingress interface, and first-match ordering can completely change the result.
- Detection logic must be tested against benign traffic as well as attack traffic; otherwise a signature may work technically while producing excessive false positives.
- Packet capture and offline replay are effective tools for separating signature logic problems from live engine state or timing problems.
- Endpoint telemetry must survive reboots and service restarts to be useful during real investigations.
- A concise incident timeline is often more useful than a large collection of screenshots because it connects evidence into a defensible conclusion.

22. Conclusion

The Enterprise Network Defense & SOC Incident Response Lab evolved from basic network monitoring into a cohesive investigation and response environment. The project connected pfSense, Snort, Wireshark, Splunk, Wazuh, Auditd, Docker, DVWA, Kali Linux, and Ubuntu Server so that activity could be observed from multiple security perspectives.

The most important outcome was the ability to move beyond alert generation. Command-injection traffic was detected on the network, validated at the packet layer, confirmed through endpoint process execution, surfaced in Wazuh, and then placed into an incident-response workflow with targeted containment and recovery. Additional SSH and FIM scenarios expanded coverage into authentication and persistence-relevant behavior, while the tuning phase demonstrated basic detection-engineering discipline.

This is an enterprise-style training lab rather than a production SOC. Its value is the repeatable analytical workflow: generate controlled activity, collect evidence, correlate independent sources, validate conclusions, respond proportionally, and tune detections based on observed false-positive behavior. That workflow is directly relevant to SOC L1 and junior Blue Team responsibilities.

A logical future extension would be to add a Windows endpoint with Sysmon and a Wazuh agent, then repeat the same evidence-driven process for Windows process creation, network connections, and persistence telemetry. This extension is intentionally identified as future work and is not presented as part of the completed evidence in this report.

Appendix A - Selected Evidence Placement Map

The following evidence set is intentionally selective. These are the screenshots recommended for the final portfolio version; duplicate or lower-value screenshots do not need to be inserted.

Figure	File	Report Section
1	08_Snort_Alert_Log_File.png	8. Network Detection with Snort
2	03_TCP_SYN_Scan.png	9. Packet Analysis with Wireshark
3	04_SQL_Injection_UNION_SELECT.png	9. Packet Analysis with Wireshark
4	05B_Command_Injection_Wireshark.png	9. Packet Analysis with Wireshark
5	16_Splunk_SOC_Dashboard.png	10. SIEM Investigation with Splunk
6	20_Splunk_Detection_Summary.png	10. SIEM Investigation with Splunk
7	24_Wazuh_Ubuntu_Agent_Active.png	11. Endpoint Monitoring with Wazuh and Auditd
8	05A_Command_Injection_Result.png	12. Incident Case Study 1
9	29_Snort_Command_Injection_Alert.png	12. Incident Case Study 1
10	28_Wazuh_DVWA_Command_Injection_Process_Chain.png	12. Incident Case Study 1
11	30_Wazuh_Command_Injection_Event_Details.png	12. Incident Case Study 1
12	32_Wazuh_SSH_Brute_Force_Detection.png	13. Incident Case Study 2
13	35_Wazuh_FIM_Create_Modify_Delete_Summary.png	14. Incident Case Study 3
14	36_Pre_Containment_Connectivity.png	15. Incident Response and Containment
15	36_pfSense_Incident_Containment_Rule.png	15. Incident Response and Containment
16	37_Post_Containment_Verification.png	15. Incident Response and Containment
17	38_pfSense_Containment_Firewall_Log.png	15. Incident Response and Containment
18	39_Post_Recovery_Connectivity.png	15. Incident Response and Containment
19	41_Snort_Tuned_POST_Detection.png	16. Detection Engineering and Tuning
20	42_Snort_Tuned_GET_No_Alert.png	16. Detection Engineering and Tuning

Not selected for the main report: duplicate Splunk source-preview images, duplicate FIM delete screenshots, repeated command-injection screenshots that show the same conclusion, and generic timestamp-named screenshots whose value is not stronger than the named evidence above.